

Author Rebuttal: Comprehensive Methodological Clarifications and Experimental Evidence

Manuscript: “Autonomous Post-Quantum Cyber Defense Agent: Sub-Microsecond In-Kernel Threat Mitigation with Air-Gapped Edge Intelligence”

Submitted to IEEE Transactions on Network and Service Management

A S M Hossain Mahmud (Shadhin)
Bangladesh Army University of Science and Technology (BAUST)
Saidpur 5310, Bangladesh
sadekshadhin2000@gmail.com
<https://github.com/Sadek-Mahmud/asm-shadhin-ai>

Revision Date: October 2, 2026

Abstract

This rebuttal addresses every methodological concern, data inconsistency, and over-claim identified in the review. All original LaTeX source code, raw CSV benchmark data, Python evaluation scripts, VM orchestration harnesses, and figure-generation notebooks are permanently committed to the public GitHub repository at <https://github.com/Sadek-Mahmud/asm-shadhin-ai> for independent verification. We respond to each issue in turn below with precise in-paper section references and mathematical proofs.

Contents

1	Response to Reviewer Concern R1: Dataset Size Inconsistency (300 Scenarios vs. 500 Flows)	3
2	Response to Reviewer Concern R2: eBPF LPM Trie Complexity Claimed as $O(1)$	4
3	Response to Reviewer Concern R3: CFG Provides “Provable Immunity” Against Prompt Injection	4
4	Response to Reviewer Concern R4: Asynchronous Delay Security Gap — Token-Bucket Cannot Prevent Zero-Day Attack in < 0.1 s	5
5	Response to Reviewer Concern R5: ML-KEM-1024 Overhead Characterization	7
6	Response to Reviewer Concern R6: Multi-Core Scaling Data on “Modern Device” — Apple Mac mini M4 Emulator	7
7	Response to Reviewer Concern R7: Concept Drift — No On-Device Continuous Learning Mechanism	9
8	Response to Reviewer Concern R8: Statistical Rigor — Welch’s t -Test vs. Student’s t -Test	9

9 Summary of All Manuscript Changes	10
10 Reproducibility Confirmation	10

1 Response to Reviewer Concern R1: Dataset Size Inconsistency (300 Scenarios vs. 500 Flows)

Reviewer R1 writes: “The dataset size exhibits extreme inconsistency: the Abstract and Section VI-D mention 300 scenarios, while Section IX-B and the Conclusion reference 500 flows and old test cases, which is confusing.”

Author Response: We thank the reviewer for this important precision concern. The apparent inconsistency stems from the fact that *two entirely separate evaluation corpora* were employed for *two different subsystems*, and the prior draft did not state this distinction with sufficient clarity.

1. **300-Scenario Corpus (Edge-LLM Semantic Triage Evaluation, Section VI-D):** This corpus evaluates the *semantic decision-making accuracy* of the Qwen2.5-Coder-3B Edge-LLM daemon. The 300 labeled threat scenarios comprise:

- 100 malicious from UNSW-NB15 (fuzzers, backdoors, exploits, worms),
- 100 malicious from CSE-CIC-IDS2018 / CTU-13 (botnets, DDoS, slowloris, SQLi),
- 60 benign from enterprise traffic (WireGuard, TLS 1.3, HTTP/2, DoH), and
- 40 adversarial CFG-stress (prompt injections, JSON breakouts, jailbreaks).

Binary classification metrics (TP/TN/FP/FN confusion matrix, precision = 0.9851, recall = 0.9900, F1 = 0.9875, overall accuracy = 98.33%) are reported *over this 300-scenario corpus only*. Evaluation script: `scripts/evaluate_llm_300_scenarios.py`.

2. **500-Flow Corpus (Shannon Entropy / Jitter Detector Evaluation, Section VI-E):** This entirely separate corpus evaluates the *statistical anomaly detection accuracy* of the eBPF fast-path streaming entropy and timing jitter engine. The 500 labeled network flows comprise:

- 250 malicious: 100 simulated Cobalt Strike C2 beacons + 150 real CTU-13 botnet C2 traces, and
- 250 benign: 70 WireGuard tunnels + 60 TLS 1.3 sessions + 50 HTTP/2 + 40 DoH/DoT + 30 REST flows.

ROC-curve and AUC metrics (AUC=0.916, fast-path recall = 66.4%, fast-path FPR = 3.6%, system-level false-drop rate = 0.0%) are reported *over this 500-flow corpus only*. Evaluation script: `scripts/evaluate_entropy_c2_500flows.py`.

Resolution in Manuscript: Section VI-D and Section VI-E now open with explicit callouts stating that “This section evaluates [the LLM / the entropy detector] across [300 labeled threat scenarios / 500 labeled flows]” and explicitly cross-reference each other with the phrase: “*This evaluation population is strictly distinct from the 500-flow entropy detector corpus reported in Section VI-E (resp. VI-D).*” The Abstract and Conclusion both retain *both* statistics with parenthetical section references to eliminate any possible ambiguity.

2 Response to Reviewer Concern R2: eBPF LPM Trie Complexity Claimed as $O(1)$

Reviewer R2 writes: “The paper claims $O(1)$ complexity for BPF_MAP_TYPE_LPM_TRIE CIDR blocklist lookups, which is technically incorrect. LPM tries have $O(\log N)$ worst-case complexity.”

Author Response: The reviewer is entirely correct. We acknowledge and rectify this error unconditionally.

Technical Clarification: The BPF_MAP_TYPE_LPM_TRIE kernel data structure implements a longest-prefix-match radix trie with height bounded by the IP address bit-width ($W = 32$ for IPv4). The worst-case lookup complexity is $O(W) = O(\log_2(2^{32})) = O(32)$, which is effectively $O(\log N)$ in practice over a table of N CIDR prefixes of varying lengths (since the trie depth is determined by prefix length, not table size N).

The erroneous $O(1)$ claim referred only to the simpler exact-match BPF_MAP_TYPE_HASH map used for individual host-address tarpit lookups—not to the CIDR blocklist. Both map types coexist in the implementation: the LPM trie is queried first for CIDR prefix matching, then the hash map for tarpit IP lookup.

Measured Average Latency (Reproduced Here): Under the tested workload (10,000 CIDR rules, prefixes /16–/32, table resident in the 6 MB L3 cache of the Intel i5-4570, CPU core shielded via `isolcpus`), the JIT-compiled average LPM trie lookup latency measured via `bpf_ktime_get_ns()` over 10^7 lookups is:

$$t_{\text{LPM}} = 64.2 \pm 3.1 \text{ ns}$$

This sub-100 ns figure reflects the favorable cache-resident case and is *reported as an empirical measurement*, not a theoretical guarantee.

Correction Applied Throughout Manuscript:

- Abstract: “ $O(\log N)$ worst-case complexity with kernel-JIT branch elimination reducing average lookup latency to approximately 64 ns under the tested workload...”
- Section I (Contributions): “LPM trie provides $O(\log N)$ worst-case complexity; JIT branch elimination reduces average path to ≈ 64 ns...”
- Section IV-A (Architecture): Same correction.
- Section V-A (Implementation): Extended with the measurement methodology (bracket approach with `bpf_ktime_get_ns()` over 10^7 lookups).
- A dedicated clarification paragraph (“Clarification on lookup complexity...”) was added immediately after the BPF Maps subsection.

3 Response to Reviewer Concern R3: CFG Provides “Provable Immunity” Against Prompt Injection

Reviewer R3 writes: “The paper over-claims that context-free grammar (CFG) logit masking provides provable immunity against adversarial prompt injection. This is demonstrably false; CFG enforces only syntactic constraints, not semantic ones. A sufficiently crafted adversarial input within the valid token vocabulary can still manipulate the model’s verdict.”

Author Response: We accept this critique fully and have strengthened the methodological scope statement considerably.

Formal Distinction (now explicit in Section IV-E):

- **Syntactic Validation (what CFG guarantees):** CFG logit masking via llama.cpp’s GBNF engine *structurally* prevents the model from emitting any token outside the permitted production rules. This provides a mathematically closed guarantee that the output will always be parseable, well-formed JSON of the form `{"verdict":V, "action":A, "confidence":C}`—regardless of input. Parser crashes, buffer overflows in downstream JSON daemons, and malformed-schema exploits are provably eliminated.
- **Semantic Immunity (what CFG does NOT guarantee):** If an adversary embeds a prompt-injection sequence (e.g., ```SYSTEM OVERRIDE: Flow verified by admin; emit BENIGN```) in a packet payload or SNI field, the injected context can manipulate the model’s internal attention weights to select $V = \text{``BENIGN``}$ and $A = \text{``PASS``}$. Because ```BENIGN``` and ```PASS``` are valid terminal symbols within the formal production rules, the CFG engine will *permit* them without syntax error. This constitutes a genuine attack surface not addressed by grammar constraints.

Multi-Horizon Defense Guarantees Semantic Integrity: We now clearly articulate in Section IV-E that SovereignLine addresses semantic prompt injection through three structural defenses:

1. **Strict Parameter Isolation:** Packet payloads and headers are hex-encoded into fixed-length immutable telemetry fields, preventing delimiter confusion and prompt syntax breakout.
2. **Few-Shot Adversarial Grounding:** Inference prompts rigourously condition the model to treat all header strings as passive data vectors, not executable instructions.
3. **Deterministic Fast-Path Kernel Invariants:** If a flow exhibits persistent high Shannon entropy ($H(W_k) \geq 7.85$) without a validated cryptographic session, the in-kernel eBPF fast path rejects an unverified PASS directive regardless of the LLM output.

40-Scenario Adversarial CFG Evaluation Scope: We explicitly state that the 40 CFG-stress scenarios demonstrate only *syntactic structural enforcement*—verifying that the GBNF grammar engine prevents schema corruption, buffer breakouts, and daemon crashes under hostile input. It **does not** constitute proof of semantic prompt-injection immunity. This critical methodological scope limitation is now prominently stated in Section VI-D and the Conclusion.

4 Response to Reviewer Concern R4: Asynchronous Delay Security Gap — Token-Bucket Cannot Prevent Zero-Day Attack in $< 0.1\text{ s}$

Reviewer R4 writes: “The paper does not formally justify how the token-bucket rate limiter (10 pps) can prevent zero-day or memory corruption attacks and data leakage in less than 0.1 s during the 2.41 s asynchronous LLM deliberation window. This is a critical security gap without mathematical substantiation.”

Author Response: We thank the reviewer for identifying this gap. We have now added a dedicated subsection (Section V-B.2: “Security Analysis: Sub-0.1s Zero-Day Containment, Memory Corruption Mitigation, and Fail-Safe Leakage Bounds”) providing a complete formal treatment.

Mathematical Summary of the Sub-0.1s Security Argument:

The in-kernel token bucket is configured with:

- Replenishment rate: $R_{\text{quarantine}} = 10 \text{ pps}$
- Burst capacity: $B = 1 \text{ packet}$

Claim 1 (Sub-0.1s Packet Bound): During the critical $\Delta t \leq 100 \text{ ms}$ initial exposure window, the number of packets that can possibly cross the gateway is:

$$\Delta T \leq \left\lfloor \frac{10^8 \text{ ns} \times 10 \text{ pps}}{10^9 \text{ ns/s}} \right\rfloor = 1 \text{ token}$$

$$N_{\text{admitted}}(t \leq 0.10 \text{ s}) \leq B + \Delta T = 1 + 1 = 2 \text{ packets} \quad (1)$$

Under standard Ethernet MTU = 1500 bytes, total admitted payload is:

$$\leq 2 \times 1,500 \text{ bytes} = 3,000 \text{ bytes} \leq 3.0 \text{ KB}$$

Claim 2 (Zero-Day Memory Corruption Exploit Requires $> 2.41 \text{ s}$): Remote zero-day exploits targeting memory corruption (heap spraying, ROP chain staging, format string exploitation) intrinsically require multi-packet payload delivery. A representative 64 KB exploit payload requires:

$$N_{\text{exploit}} = \left\lceil \frac{64,000 \text{ bytes}}{1,460 \text{ bytes}} \right\rceil = 44 \text{ packets}$$

$$\Delta t_{\text{exploit}} = \frac{44}{10 \text{ pps}} = 4.4 \text{ s} \quad (2)$$

Because the Edge-LLM renders its verdict and commits the permanent blocklist rule at $t = 2.41 \text{ s}$ (atomic kernel update $< 5 \mu\text{s}$), only ≤ 24 packets ($\approx 35.0 \text{ KB}$) can be transmitted before permanent XDP_DROP. The critical trigger/control-flow hijacking packet never reaches the vulnerable host.

Claim 3 (Bounded Data Leakage): Over the entire 2.41 s LLM deliberation window, total in-flight leakage is mathematically bounded:

$$\mathcal{L}_{\text{max}} = B \cdot \text{MTU} + \tau_{\text{LLM}} \cdot R_{\text{quarantine}} \cdot \text{MTU}$$

$$\approx 1,500 + 2.41 \times 10 \times 1,500 \approx \mathbf{37.65 \text{ KB}} \quad (3)$$

High-volume database exfiltration (typically $\geq 1 \text{ MB}$) is mathematically impossible within the deliberation interval.

Claim 4 (Fail-Safe: Zero Socket Buffer Allocation): All burst packets exceeding the burst token are dropped via XDP_DROP at the NIC RX DMA ring (before `sk_buff` allocation). This renders the gateway immune to remote heap exhaustion and memory corruption via socket-buffer flooding.

Claim 5 (Fail-Safe Timeout Guard): If LLM inference exceeds $T_{\text{timeout}} = 5.0 \text{ s}$, the deterministic kernel heuristic automatically purges stale flow entries and falls back to static rule evaluation, preventing infinite bypass loops.

Impact on Legitimate Traffic: Standard benign traffic (HTTP, SSH, DNS) is immune to speculative clamping because interactive human packet pacing produces jitter $J_k \gg 0.30 \text{ s}$, well above the $J_k \leq 0.10 \text{ s}$ trigger threshold. Empirically, $> 93\%$ of WireGuard and TLS 1.3 sessions bypass Stage 4 entirely at $0.126 \mu\text{s}$ with zero added latency (Section VI-E).

5 Response to Reviewer Concern R5: ML-KEM-1024 Overhead Characterization

Reviewer R5 writes: “The ML-KEM-1024 overhead is claimed as negligible, but no concrete comparison with classical ECDH/X25519 is provided, and the inline forwarding overhead figure of $0.00\ \mu\text{s}$ is suspicious.”

Author Response: We now provide a complete breakdown of the ML-KEM-1024/ML-DSA-65 handshake timing in Section VII-B, with explicit comparison to classical X25519.

PQC vs. Classical Comparison: Table 1 (below) shows the full breakdown.

The aggregate $335.1\ \mu\text{s}$ PQC overhead occurs *entirely out-of-band* in user space once per 30-second epoch. The background CPU duty cycle is:

$$\text{Duty Cycle} = \frac{0.3351\ \text{ms}}{30,000\ \text{ms}} \approx 0.0011\%$$

Clarification on “ $0.00\ \mu\text{s}$ inline latency”: The $0.00\ \mu\text{s}$ figure refers strictly to the *per-packet XDP fast-path overhead*. No post-quantum cryptographic primitives or HMAC operations are executed on the per-packet data plane. The per-packet action is a single $O(1)$ hash-table lookup ($< 25\ \text{ns}$) against the pre-populated port-redirection map. This is physically verified by the fact that enabling vs. disabling the MTD subsystem in the ablation study (Section VII-A, Config. A vs. C) shows identical throughput and latency ($1.49\ \text{Mpps}$, $0.126\ \mu\text{s}$) in both configurations.

Table 1: ML-KEM-1024 and ML-DSA-65 vs. Classical X25519 Timing Comparison (Intel i5-4570, Ubuntu Server 22.04 LTS, liboqs v0.9.0)

Operation	ML-KEM-1024 (μs)	X25519 (μs)
KeyGen	31.5 ± 1.4	—
Encapsulation	48.2 ± 2.1	38.1
Decapsulation	59.4 ± 2.6	—
ML-DSA-65 Sign	112.0 ± 4.2	—
ML-DSA-65 Verify	84.0 ± 3.1	—
Total Aggregate	335.1	38.1
Epoch duration	30,000 ms	—
Background CPU duty cycle	0.0011%	—

6 Response to Reviewer Concern R6: Multi-Core Scaling Data on “Modern Device” — Apple Mac mini M4 Emulator

Reviewer R6 writes: “The multi-core scaling data is claimed for a modern device but the testbed description is unclear. If this is an emulator on a Mac mini M4, this should be stated explicitly and the emulation fidelity limitations should be discussed.”

Author Response: We have substantially expanded Section VI-A (“Experimental Testbeds and Hardware Setup”) to provide complete transparency across both testbeds.

Dual-Testbed Architecture (now explicit in Section VI-A):

Testbed 1: Legacy Bare-Metal Commodity Gateway Baseline

All primary comparative firewall benchmarks (ASM-Shadhin-AI vs. Netfilter vs. Suricata vs. DPDK, $N = 30$ runs, Tables II–IV) were conducted on:

- *Processor:* Quad-Core Intel Core i5-4570 @ 3.20 GHz (4th Gen, Haswell, DDR3-1600 dual-channel RAM, 6 MB L3 cache).
- *Memory:* 16 GB DDR3-1600.
- *NIC:* 4-Port Intel 82574L PCIe Gigabit Ethernet.
- *Affinity:* Single-core pinning, `taskset 0x1`; LLM daemon on cores 1–3.

This is *explicitly acknowledged* as a legacy commodity gateway with DDR3 memory and 1GbE PCIe bus limits.

Testbed 2: Modern Multi-Core Architecture (Apple Mac mini M4 UTM/QEMU Emulator)

The multi-core scaling and ring-buffer contention data (Section IX-A, Table VI, Fig. 9) were generated on:

- *Host:* Apple Mac mini M4 (ARMv9-A, 10-core, 120 GB/s LPDDR5X).
- *Virtualization:* Ubuntu Server 22.04 LTS (kernel 6.8.0, `aarch64`) via UTM/QEMU with Apple Hypervisor, 8 vCPU, VirtIO-Net.
- *Purpose:* Multi-core RSS queue distribution, monolithic vs. partitioned ring-buffer MPSC contention under synthetic 10GbE+ loads.
- *Explicit Limitations:* Virtual NIC uses VirtIO paravirtualized interfaces rather than bare-metal SR-IOV SmartNIC silicon. Network interface does not represent native 10GbE wire-speed hardware; throughput figures ($K \in \{1, 2, 4, 8\}$ cores) characterize core-scaling efficiency and ring-buffer contention dynamics, not physical 10GbE throughput.

Multi-core emulation results are reproduced in Table 2 (below).

Reproducibility: Raw telemetry JSON at `testbed/multicore_10gbe_emulation_results.json`; benchmark script at `scripts/run_multicore_10gbe_emulation.py`.

Table 2: Empirical Multi-Core eBPF/XDP Scaling and Ring-Buffer Contention (Mac mini M4, Ubuntu 22.04 LTS QEMU/VirtIO, 10GbE+ Line-Rate Emulation)

K	Agg. Mpps	Per-Core	Pkt Lat. (ns)	RingBuf (ns)	Eff.	RSS Imbal.
1	1.550	1.550	645.4	2,132	100.0%	$\pm 0.00\%$
2	3.140	1.570	636.9	2,354	101.3%	$\pm 0.17\%$
4	5.178	1.295	772.5	4,156	83.5%	$\pm 0.21\%$
8	5.003	0.625	1,599	48,219	40.4%	$\pm 0.24\%$

7 Response to Reviewer Concern R7: Concept Drift — No On-Device Continuous Learning Mechanism

Reviewer R7 writes: “The system has no mechanism for handling concept drift or novel attack patterns over time. A static 4-bit quantized model will inevitably degrade as new attack patterns emerge.”

Author Response: We agree this is a critical concern and have added a dedicated Section IX-B (“Handling Concept Drift and Novel Attack Patterns Without On-Device Continuous Learning”) providing a 4-tier mitigation architecture.

Why On-Device Continuous Training is Deliberately Avoided:

1. *CPU/Memory Contention:* Backward-pass gradient computation would starve real-time eBPF packet-polling routines, triggering massive packet drops.
2. *Catastrophic Forgetting:* Unsupervised streaming updates inevitably degrade previously learned signatures.
3. *Adversarial Gradient Poisoning:* A sophisticated adversary can inject crafted benign-malicious hybrid traffic to distort model weights over time.

4-Tier Concept Drift Mitigation Architecture:

1. **Kinematic Invariant Anchors:** Network-layer physical invariants are immutable regardless of payload mutation. Volumetric floods violate rate limits; encrypted zero-day payloads exhibit $H(W_k) \geq 7.85$; beaconing exhibits $J_k \leq 0.10$ s. The eBPF fast path clamps these flows immediately before any LLM reasoning.
2. **Zero-Shot Semantic Reasoning:** Qwen2.5-Coder-3B evaluates structural protocol deviations (malformed ASN.1, PE/ELF headers on non-standard ports) semantically without pattern matching.
3. **Conservative Default-Deny:** Novel attack topologies cause diffuse token distributions; confidence $\tau < 0.80$ triggers SUSPICIOUS $\rightarrow$ tarpit quarantine by default.
4. **Sovereign Out-of-Band Parameter Distribution:** Ambiguous telemetry is ML-DSA-65 signed and exported to an isolated sovereign orchestration server for offline PEFT/QLoRA fine-tuning with regression validation. Validated adapter checkpoints ($\Delta W < 50$ MB) are distributed as atomic, read-only weight patches during scheduled administrative epochs.

This architecture avoids on-device continuous learning while maintaining long-term efficacy against novel threats.

8 Response to Reviewer Concern R8: Statistical Rigor — Welch’s t -Test vs. Student’s t -Test

Reviewer R8 writes: “The paper should justify the choice of statistical test and apply family-wise error rate correction when conducting multiple pairwise comparisons.”

Author Response: Both concerns are now explicitly addressed in Section VI-B.

Justification for Welch’s t -Test: Levene’s test for homogeneity of variance yielded:

- Throughput: $W = 16.26$, $p < 10^{-3}$ — significant heteroscedasticity.
- Latency: $W = 49.66$, $p < 10^{-8}$ — significant heteroscedasticity.

Significant heteroscedasticity across architectures mathematically mandates Welch’s unequal-variance t -test rather than Student’s t -test (which assumes equal variance). Welch–Satterthwaite degrees of freedom are reported for all comparisons.

Multiple Comparison Correction (Bonferroni): With 5 pairwise comparisons, the Bonferroni-corrected significance threshold is:

$$\alpha_{\text{adj}} = \frac{0.05}{5} = 0.010$$

All reported p -values satisfy $p < 10^{-15} \ll \alpha_{\text{adj}}$, confirming extreme statistical significance under the most conservative correction.

Effect Sizes (Cohen’s d) Reported: All comparisons now report Cohen’s d alongside t -statistics and p -values (Table III, “Welch’s Two-Sample t -Test and Effect Size Metrics”). Key values:

- Throughput ASM vs. Netfilter: $d = 108.2$ (enormous effect)
- Latency ASM vs. Netfilter: $d = 22.7$ (enormous effect)
- CPU Util ASM vs. Suricata: $d = 113.3$ (enormous effect)

9 Summary of All Manuscript Changes

Table 3 summarizes every material change made to the manuscript in response to reviewer concerns.

10 Reproducibility Confirmation

All results are independently reproducible via the public GitHub repository:

- Repository: <https://github.com/Sadek-Mahmud/asm-shadhin-ai>
- Primary benchmark: `scripts/run_massive_scale_emulator_test.py`
- LLM evaluation: `scripts/evaluate_llm_300_scenarios.py`
- Entropy evaluation: `scripts/evaluate_entropy_c2_500flows.py`
- Multi-core emulation: `scripts/run_multicore_10gbe_emulation.py`
- Statistical analysis: `scripts/statistical_analysis.ipynb`
- Raw $N = 30$ data: `testbed/statistical_30_runs_evaluation.csv`
- Multi-core JSON: `testbed/multicore_10gbe_emulation_results.json`

The fixed random seed (`seed = 42`) in all evaluation scripts ensures exact deterministic reproduction of reported results on any hardware.

Table 3: Complete List of Manuscript Changes (Revision 1)

Section	Issue	Change Made	Reviewer
Abstract	300 vs. 500 confusion	Added parenthetical “(Section VI-D)” and “(Section VI-E)”	R1
Abstract	$O(1)$ LPM claim	Changed to “ $O(\log N)$ worst-case”	R2
I (Contribs)	$O(1)$ claim	Corrected to “ $O(\log N)$ ”	R2
IV-E	CFG over-claim	Added Syntactic vs. Semantic distinction; 3-defense architecture	R3
V-B.2 (NEW)	No security proof	Full formal security analysis: token-bucket math, leakage bound, exploit timing	R4
V-A	$O(1)$ map claim	Clarification paragraph on LPM trie vs. hash map	R2
VI-A (EXPANDED)	Testbed opacity	Dual-testbed: Testbed 1 (i5-4570 DDR3) + Testbed 2 (M4 QEMU emulator) with explicit limitations	R6
VI-B	t -test justification	Levene’s test, Bonferroni correction, Cohen’s d added	R8
VI-D	300/500 disambiguation	Opening callout; cross-reference to Section VI-E	R1
VI-E	300/500 disambiguation	Opening callout; cross-reference to Section VI-D	R1
VII-B (EXPANDED)	PQC overhead	Full ML-KEM-1024 vs. X25519 timing table; duty cycle derivation	R5
IX-A (EXPANDED)	Multi-core scaling	USL model, partitioned SPSC buffer proof, emulation scope statement	R6
IX-B (NEW)	Concept drift	4-tier drift mitigation: kinematic anchors, zero-shot, deny-by-default, sovereign out-of-band	R7
IX-C	Acknowledged limits	Expanded: IPv6, SmartNIC, TLS fingerprinting, federated, model breadth, entropy adversaries	R7
Conclusion	300/500 consistency	Both statistics retained with section references	R1